

Deep Learning — Assignment 13

Assignment for week 13 of the 2023 Deep Learning course (NWI-IMC070) of the Radboud University.

Names: Vojko Pjanovic (s1084416); Carmen Gascó Gálvez (s1098566)

Group: 10

Instructions:

- Fill in your names and the name of your group.
- Answer the questions and complete the code where necessary.
- Keep your answers brief, one or two sentences is usually enough.
- Re-run the whole notebook before you submit your work.
- Save the notebook as a PDF and submit that in Brightspace together with the `.ipynb` notebook file.
- The easiest way to make a PDF of your notebook is via File > Print Preview and then use your browser's print option to print to PDF.

Objectives

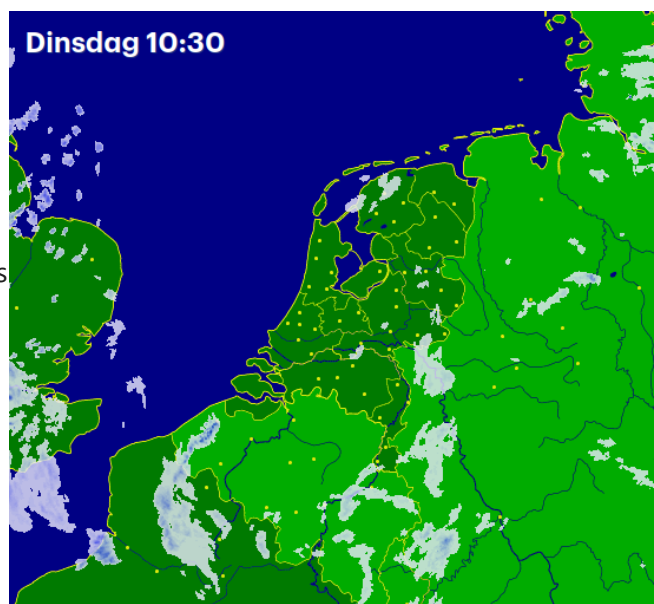
In this assignment you will

1. Think about the design of deep neural networks on a higher level
2. Read papers that use deep learning in an application

This week's assignment does not involve any programming.

13.1 Predicting the weather (7 points)

Most people don't like to be outside in the rain. Fortunately, we nowadays have radar sensors, that can show the amount of precipitation over a wide area in real time. Weather services like [buienradar](#) use this data to forecast when it will rain. But the models they use are often quite simple: just move the rainclouds



according to the current wind patterns.

In this assignment you will investigate whether you can do better with deep learning.

More concretely: The task that we have in mind is to predict the amount of rain up to 3 hours in advance, for the entire country of the Netherlands. The input will be radar images of the amount of rain *currently* falling. For training, there is historical radar data going back several years. You may assume that the data has been gathered with a sample rate of 5 minutes, and at a resolution of 1500m per pixel. See [the KNMI website](#) for more information.

(a) Are there risks of unfair biases in this task? Explain your answer. (1 point)

As always, some biases exist.

The main bias would be that it is based on historical data, and the weather is a stochastic system, portraying chaotic behaviour which is not found in previous historical data.

Moreover, the data collection system is also somewhat biased, given that some parts of the country are more densely populated than others, which would require more accurate measurement for accuracy per person than sparsely populated parts.

(b) Suppose we want to use a deep neural network for this task. How would the input data be represented? (1 point)

For now, consider using only the current radar data as input.

So the main input is the current radar data. This would be a snapshot of the rainfall in a set unit - i.e. rainfall in mm - , at all locations. First of all, this snapshot would probably be processed to normalize over all snapshots, and smooth between the recording locations. We can resize the rainfall per locations to the physical locations in the Netherlands as well.

Consequently, we want to add in both the temporal and spatial features, which account for the dynamics. This can be done with some metadata such as position coordinates and timestamps.

(c) What are the targets for prediction? How are they represented? (1 point)

The target of prediction was the amount of rain up to 3 hours in advance for the entire country of the Netherlands. This would be in the same format as the input data, i.e. the amount of rainfall in mm per location in a grid-like structure output, but it would be accumulated over the 3 hours.

(d) If we want to make a prediction for the entire country of the Netherlands, how large should the input be to contain all the relevant information? (1 point)

```
In [ ]: import numpy as np

entire_area = 41850 # in km^2
```

```
area_per_pixel = 1.5*1.5 # in km^2
total_pixels = entire_area/area_per_pixel
print(f'total pixels input: {total_pixels}')
```

total pixels input: 18600.0

The above calculation illustrates how many pixels are needed as input given the total area of the Netherlands and the pixel resolution. This could be resized to a MxN matrix to accommodate for the spatial component of the surface of the Netherlands. Of course, if other (meta)data is included, the input would be bigger, added to different channels.

Rainclouds act differently depending on the terrain. So it can be useful for the model to know which areas of the radar image are above sea, rivers, forests, cities, mountains (as if), etc.

(e) How can you include this information in the input to the model? (1 point)

In this case, that could be added as data to every pixel from satellite images of the Netherlands. Then we could account for the rainfall together with the terrain type. This would likely be included as separate channels to the input data.

(f) If information on the type of terrain is not available, could it be learned instead? If so, how? (1 point)

It could be inferred from satellite images as mentioned above. However, this would require some existing pre-trained model, with transfer learning or pre-trained CNNs, or at the very least labelled data of satellite images would be needed to classify the NL terrain. Otherwise things would become rather complicated.

The weather also depends on the time of day, in particular on whether the sun is shining. And on the time of year, especially the temperature.

(g) How could the time of day and time of year be given as inputs to the network? (1 point)

The time of day and year would be added as general metadata for every snapshot. This would be another datapoint, turning x into $x = \{\text{'time': ...}, \text{'data': } M \times N \times 2\}$ where the time entry is a single value in unix time, and the data entries are 2 MxN matrices, one containing rainfall amount, and the other the type of terrain. Of course, using a dictionary is most probably not very efficient, but it should clarify the structure of the data.

13.2 Weather prediction models (4 points)

Consider a model that uses only the current radar data and terrain as input, to predict rain 3 hours in the future.

(a) What kind of model would you use for this task? (1 point)

Since we are neglecting the temporal component - which we would represent with a RNN/LSTM - we only look at the spatial component of the input, which would best be modelled using a CNN.

(b) How would the performance of a model that takes only a single radar image as input compare against the simple baseline model that moves the radar image in the wind direction? (1 point)

It would most probably be worse. Maybe the movement could be inferred from the patterns in the clouds, which might indicate in some way the previous wind and from this follows likely wind directions, but generally this would be quite hard.

Having the current wind direction would be very beneficial for predicting future rain movements.

To improve the model, it makes sense to take the temporal aspect into account, by including historical data. So radar scans from 5 minutes ago, 10 minutes ago, etc.

(c) Give two ways to include this additional data into the model. (2 points)

To include this additional data, one would have to implement other models which are able to account for the temporal components.

Two commonly used architectures which allow for learning temporal components are sequential recurrent neural networks, and temporal CNNs. These allow for including the additional data into the model.

13.3 MetNet (4 points)

We are not the first people to think about this problem. The paper [MetNet: A Neural Weather Model for Precipitation Forecasting](#) has tried to tackle the task of predicting rain from radar images.

(a) Have a look at the MetNet paper and compare their method to your answers in 13.1 and 13.2. What are they doing the same, what are they doing differently? (3 points)

Input encoding: Both MetNet and the recommended approach use radar and satellite data as input, represented in a multi-dimensional tensor, although MetNet additionally includes the target time as input.

Output encoding: MetNet and the recommended approach share similarities in output encoding, as both aim to predict weather conditions at a target time.

Model: MetNet and the recommended approach differ slightly in their model architectures; while both utilize convolutional neural networks (CNNs) for spatial processing, MetNet employs a convolutional LSTM for temporal encoding and axial self-attention blocks for spatial aggregation, whereas the recommended approach suggests

using sequential recurrent neural networks or temporal CNNs for temporal modeling alongside CNNs for spatial processing.

(b) Can the trained MetNet be used in the Netherlands using data from KNMI, without retraining? How? Or why not? (1 point)

It probably could not be used. The Netherlands have quite a different climate than the USA, on which the model is trained, so the trained data would be quite different from the required data. This would come to a loss of accuracy. However, the *type of data* is the same - mm of rainfall per area - , so it could still be run on data from the Netherlands. It is just that the underlying dynamics of the system are probably different enough.

An analogy would be image recognition: If one has trained a model to identify cats, but consequently wants to identify dogs, the task is exactly the same but the data is rather different. We think that similarly here the data is sufficiently different for the model to not return great results.

13.4 MetNet - version 2 and 3 (4 points)

You may have noticed that the MetNet paper came out 3 year ago (you can tell the year from the `/YYMM.NNNNN` arxiv url). After that two new models have been released by the same group:

- [MetNet-2: Skillful Twelve Hour Precipitation Forecasts using Large Context Neural Networks](#)
- [MetNet-3: Deep Learning for Day Forecasts from Sparse Observations](#)

(a) What method is used by MetNet-2 to get a larger receptive field in the convolutions? (1 point)

Dilation factors in the CNN

(b) How many parameters do the original MetNet and MetNet-3 have? Is one model significantly larger than the other? (1 point)

MetNet-1 has 225M trainable parameters MetNet-3 has 227M trainable parameters

Even though the parameters are of equal magnitude, it is still possible, and very likely, that MetNet-3 is still larger/more complex. Metnet-3 probably has wider/deeper layers, and more complex architecture.

(c) Estimate how much CO₂ was emitted for training MetNet-3. (2 points)

Hint: You can find [information on the power consumption of google TPUs here](#) (the reported numbers are per chip, not per pod).

Hint 2: Make a reasonable assumption for the CO₂ / kWh (see lectures, or [here](#))

The values on the website are per pod, and each pod contains two cores. In the paper they mention usage of 512 TPUv3 cores, so the power usage in the Google table correspond to two cores in the paper.

```
In [ ]: tpus = 512/2 # TPUv3 cores, as power output is for chips, and there are 2 cores
total_time = 7*24*60*60 # 7 days in seconds
mean_power = 220 #Watts per TPUv3 chip, watts = joules/second

total_power = tpus*mean_power*total_time # for all cores, in joules
total_kwh = total_power/1000/3600 # in kWh
gc02perkwh = 370 #gCO2/kWh in North America, given that the paper is centered ar
total_co2 = total_kwh*gc02perkwh # total co2 in grams

print(f'total co2 (in tonnes): {(total_co2 * 1e-6):.3f} tonnes')
```

total co2 (in tonnes): 3.501 tonnes

```
In [ ]: car_kms = total_co2/215 # how many kms a 2022 renault clio should drive to emit
print(f'car kms: {car_kms:.3f} kms (2022 renault clio)')
```

car kms: 16283.029 kms (2022 renault clio)

The end

Well done! Please double check the instructions at the top before you submit your results.

This assignment has 19 points.

Version 6262555 / 2023-12-11